

Deep Dive: Apache, ModSecurity & Log Triage

The upstream breach case, solved together — the two new challenges stay yours.

Stefanus Eko Prasetyo

Lab-4 · companion to the Lab 04 lecture

What we cover today

1 Log triage recipes

awk/sort/uniq patterns that turn 1,488 lines into 5 findings

2 The audit log, section by section

A/B/C/F/E/H - and how to correlate transactions to access lines

3 CRS and anomaly scoring

what the rule IDs mean and how 980130 adds up the verdict

4 Decode chains

URL, double-URL, base64, char() - payloads in layers

5 Worked case + your turn

the breach, end to end; then Challenges B and C

Five one-liners that find the attacker

root@dfir:/var/log/apache2#

```
$ awk '{print $1}' access.log | sort | uniq -c | sort -rn
  38 192.168.0.106      # everyone else is noise-free
$ awk '{print $9}' access.log | sort | uniq -c
$ awk -F'"' '{print $6}' access.log | sort | uniq -c
# $6 between the quotes = user agent
$ awk '$9==404' access.log | wc -l      # scan bursts
$ awk '{print $10}' access.log | sort -rn | head -3
 96234 / 41877      # the two dumps, by size
```

Why these five

- ▶ IP ranking: who is disproportionate
- ▶ status mix: 404 bursts = content discovery
- ▶ UA ranking: tools announce themselves
- ▶ size ranking: the biggest answers are exfiltration
- ▶ cut/awk field numbers refer to the line layout

Correlating audit transactions to access lines

root@dfir:/var/log/apache2#

```
# find the transaction near a suspicious time:
$ grep -n "01:37:49" modsec_audit.log | head
[16/Feb/2023:01:37:49.214415 +0500] Y-1C...

# the transaction id ties every section together:
--0f83dc52-A-- ... --0f83dc52-Z--

# pull one transaction cleanly:
$ sed -n '/--0f83dc52-A--/,/--0f83dc52-Z--/p' modsec_audit.log
```

Correlation keys

- ▶ timestamp (audit has microseconds)
- ▶ transaction id + unique_id in H
- ▶ client port in A matches nothing in access.log - use time
- ▶ Stopwatch micro-time orders events inside one second

Anomaly scoring — the 980130 verdict

```
root@dfir:/var/log/apache2#
```

```
Message: ... [id "980130"] [msg "Inbound Anomaly  
Score Exceeded (Total Inbound Score: 43 -  
SQLI=0,XSS=0,RFI=0,LFI=35,RCE=5,  
PHPI=0,HTTP=0,SESS=0)  
  
# each matched rule adds points to a category;  
# 980130 fires when the total crosses the threshold  
# LFI=35 = five-ish traversal rules matched at once
```

Reading the verdict

- ▶ the category split names the attack classes
- ▶ severity CRITICAL + score = confidence
- ▶ DETECTION_ONLY = points but no block - check H footer
- ▶ paranoia levels: higher = more rules, more noise

Signature grep table

Hunt for	Pattern
Traversal	<code>grep -E '\.\.\/ %2F\.\.'</code> access.log
RCE verbs	<code>grep -E 'cmd= whoami cat\+ %2Fetc'</code> modsec_audit.log
SQLi	<code>grep -Ei "union select '-- 1=1 libinjection" *</code>
Tool UAs	<code>grep -Ei 'sqlmap dirb nikto gobuster nmap'</code> access.log
Rule IDs	<code>grep -oE '\[id "[0-9]+\"]'</code> error.log sort uniq -c
Errors that matter	<code>grep -E 'Permission denied Fatal error'</code> error.log

Decode chains you will actually need



root@dfir:/var/log/apache2#

```
# 1. URL decode (once or twice):  
$ python3 -c "import urllib.parse as u; print(u.unquote_plus(S))"  
  
# 2. base64 blobs:  
$ echo 'ZmxhZ3toMXBw...' | base64 -d  
  
# 3. SQL char() sequences -> ASCII:  
$ python3 -c "print(''.join(chr(c) for c in [90,109,120]))"  
Zmx  # then recognize base64 and decode again
```

Recon, then the traversal ladder



root@dfir:/var/log/apache2#

```
# Q1: one IP dominates:
192.168.0.106    (38 requests, Firefox 102 on Ubuntu Linux X11)

# Q2 (part 1): the ladder - 01:35:07 to 01:35:30
/images.php?file=%2Fetc%2Fpasswd  -> direct try, bounces to view.php
/view.php?image=../etc/passwd ... ../..../etc/passwd  200, 650B
/view.php?image=../..../etc/shadow  200, 202B  # blocked!

# Q5: shadow DENIED - error.log:
PHP Warning: file_get_contents(...etc/shadow): Permission denied
# server runs as www-data: least privilege saved the hashes
```

RCE: the commands, the database, the note

root@dfir:/var/log/apache2#

```
# C-sections of four POSTs to /command.php:
cmd=whoami          -> www-data
cmd=ls+-la+/        -> database.sql, important_note.txt!
cmd=cat+%2Fdatabase.sql
    -> 5 users (emails + passwords), DB user + $up3rs3cr3tp4ssw0rd
cmd=cat+%2Fimportant_note.txt
    "we don't want the world to know that our password is
    'sup3r_s3cr3t_4nd_1mp0rt4nt_p4ssw0rd'"    # Q7 secret
# Q6: 5 user records + system users + 2 secrets compromised
```

The lesson

- ▶ ls -la / found the loot in the first place
- ▶ the note was also reachable via traversal
- ▶ WAF saw everything - and blocked nothing

SQLi, sqlmap, and the taunt

root@dfir:/var/log/apache2#

```
# Q4: automated tool - the UA confesses:
"sqlmap/1.6.11#stable (https://sqlmap.org)" + random probes
  search=H0vh / fMgn      # sqlmap fingerprinting

# Q8: the message to the admin (a POST body):
search=' union select 'Gentlemen, it is with great
  pleasure I inform you that:', concat(char(90),
  char(109), ...) , ':D' #

# decode chain: unquote_plus -> chr() join -> base64:
"Gentlemen..." + the base64 string + :D
```

Upstream flag (original format):

```
flag{h1pp1ty_h0pp1ty_y0ur_w3bs1t3_
1s_n0w_my_pr0p3rty!}
```

The chain

- ▶ URL decode the body
- ▶ chr() codes build base64
- ▶ base64 -d reveals the flag

Indicators of attack vs compromise (Q9-Q10)

Category	Indicator from this case
Atomic IOC	source IP 192.168.0.106; UA 'sqlmap/1.6.11#stable'
Behavioral IOA	404 bursts, traversal ladder, boolean probe pairs, UNION SELECT
WAF evidence	rule IDs 930100/110/120, 932160, 942100 at CRITICAL; score 43 (LFI=35, RCE=5)
Confirmed compromise	200 responses with passwd content, database.sql read, note read
Failed attempts	/etc/shadow (twice) - permission denied, logged in error.log
Detection rule to build	alert on 980130 + same-IP 404 burst > 30 in 5 minutes

The whole breach, one table

Time (+0500)	Event	Evidence
01:34:53	recon browsing: home, images.php	access.log lines 1-4
01:35:07-27	path traversal ladder to /etc/passwd	access.log 5-10; rules 930100/110
01:35:30	/etc/shadow attempt - DENIED	error.log Permission denied
01:35:39-47	RCE: whoami, ls, cat database.sql	audit C-sections
01:36:02-07	404 probes: d.php, database.php	access.log 17-18
01:36:19-31	manual SQLi boolean probes	audit bodies
01:36:55+	sqlmap automated SQLi	UA in access.log
01:37:16-25	more RCE: cat /important_note.txt	audit C/E
01:37:49	traversal reads important_note.txt	access.log 32-33
01:38:28	final SQLi UNION dump	audit E-section

Common mistakes, proven habits

Mistake	Habit instead
Reading only access.log	POST bodies and responses live in the audit log - always open section C and E
Decoding payloads once	decode until the string stops changing; layers hide layers
Trusting status 200 as success	200 + small size can be an error page; check E and byte count
Editing logs in place	work on copies from logs.zip; hash before and after
Reporting 'hacked' without outcome	separate attempted (IOA) from successful (IOC) in every report

Two fresh log sets. Same discipline.

01

Challenge B — 4.2

traffic_shop.log: 1,488 lines of shop traffic with one bad evening. Rank the IPs, catch the scanner by its user agent and its 404s, watch the probe pair and the two UNION dumps, then decode the goodbye query - carefully, and more than once.

02

Challenge C — 4.3

waf_alert.log: four audit transactions. Map the sections, find which one actually exfiltrated (the E section decides), name the rules that fired, and explain why the WAF watched everything and stopped nothing.

03

Working paper

pre-lab first; staged hints in section 4; evidence numbering B-xx / C-xx; quote the exact log line for every claim.